# Robot Guidance Challenge Final Report

Name: Joshua Savunthararajah

Student ID: 501181554

Section: 14

COE538: Microprocessor Systems

Date Submitted: November 25, 2024

**Project Description**

The project involved using HCS12 assembly language to create a complete control program for the EEBot robot. Sensor reading, motor control, LCD display handling, timer interrupts, and a state-machine-based navigation system were all implemented in the final program. The code's main functional blocks contained a number of important sections. All of the project's hardware modules, such as the ADC (ATD) for reading line sensors and battery voltage, ports for motor direction/enable pins, the LCD for showing system status, and the Timer Overflow Interrupt to enable timed turns, were configured by the Initialization Routines. In order to guarantee that each subsystem began in a known, stable state, these routines also initialized internal variables and buffers.

The five optical sensors (line, bow, mid, port, and starboard) were controlled by the Sensor Reading System. Activating the guider LEDs, choosing each sensor via PORTA, waiting for stabilization, initiating an ADC conversion, and storing the outcome in RAM were the steps. This block made sure that the robot's position in relation to the line was always up to date. The project's central logic was the State Machine (Dispatcher). The robot's behavior was broken into several states: START – waits for bumper release, FWD – moves forward while following the line, LEFT_TRN / RIGHT_TRN – stronger corrective turns, LEFT_ALIGN / RIGHT_ALIGN – small position adjustments, REV_TRN – reverses & turns if the front bumper is hit, and ALL_STOP – halts completely when the rear bumper is pressed. Depending on the robot's conditions, the dispatcher directed execution to the appropriate state routine. Because of its modular design, the robot's behaviour was predictable and simpler to troubleshoot.

Short routines such as INIT_FWD, INIT_LEFT, INIT_RIGHT, INIT_REV, and INIT_STOP made up the Motor Control Routines. These were in charge of setting motor direction bits, turning on and off the drive motors, and limiting turning time using the TOF counter. The robot was able to control its direction smoothly thanks to these routines. The battery voltage, which was converted from an ADC reading, scaled, and converted to BCD –>ASCII, as well as the current state name, were displayed on the LCD Display System. The robot's internal state could be monitored in real time thanks to separate routines that handled writing characters, writing strings, clearing the display, and turning numbers into readable text. Lastly, the Timer Overflow Interrupt allowed for precise timed behaviour without interfering with the main loop by providing a periodic interrupt that increased a counter used for controlling delays and timing turns.

**What the Completed Project Accomplished**

The completed program allowed the EEBot to use its five optical sensors to follow a line and to adjust its course when it drifted by turning left or right. When only slightly off-center, it could

react to collisions and realign itself. Pressing the rear bumper would result in a complete stop, while the front bumper would perform a reverse + turning recovery. The robot ran continuously in a closed-loop control system and showed real-time data on the LCD, such as battery voltage and behaviour state. With the help of user feedback, obstacle handling, and automatic self-correction, the robot was able to move forward steadily overall.

**Main Problems Encountered and How They Were Solved**

One issue was that the raw ADC values were initially inconsistent, making it difficult to interpret sensor readings. The robot's behaviour became much more stable by establishing baseline values for each sensor, adding variances to make alignment detection more tolerant, and comparing readings using a structured method. Complex state-machine debugging was another problem; early on in the development process, numerous nested comparisons made the state logic unclear. This was resolved by separating alignment logic from bumper logic, adding clear labels and a consistent branching structure, and rearranging the code into small, dedicated state routines. This improved readability and made debugging much easier.

There were issues with the LCD display because it would sometimes display characters that were corrupted. The LCD became dependable and user-friendly by implementing an appropriate E-pulse routine, adding brief delays after each LCD write, and using buffers to write entire lines. When motors first spun incorrectly or failed to start, motor direction problems occurred. This was resolved by using the TOF timer to test particular timed turns, which stabilized movement behaviour, isolating motor routines for clarity, and confirming the correct PORTA bit mapping. Finally, the lack of a compiler to detect errors, numerous interdependent subsystems, and low-level hardware control made managing a large project in assembly difficult.

The most important lessons were that small routines are easier to test than large code blocks, that organization is important, and that the best way to control robot behavior is with a state machine. Debugging one subsystem at a time was essential. If I were working on a different project, I would create a debugging checklist before integrating everything, plan the code structure earlier, and start with smaller, modular routines. The dispatcher-based state machine, the modular design, and the use of buffers for LCD output were all effective. The sensor comparison logic, making sure the motor direction bits matched the hardware, and preventing unintentional infinite loops in assembly were challenging.

**Final Remarks**

This project required integrating sensors, motors, interrupts, and displays into a single coordinated system. Despite the challenges of working in assembly language, the final robot behaved reliably and showed how powerful a well-organized state machine is.